Code guide: the fitting layer, chunk by chunk

C. H. Ross. 2026-11-05. Distributed with the package source.

1. Scope

This chapter walks through the code that takes a detection flatfile and returns a ranked set of fitted detection functions. It follows the order in which `sweep_models()` runs, and stops at each chunk of code to state what the chunk does, what it reads and writes, and why it is written the way it is.

It covers the fitting path (`prepare_distance_data()`, `model_set()`, `sweep_models()` and the selection table), the pre-flight checks (`diagnose_sweep()`, `detection_structure()`), the plotting method (`effect_estimates.ddf()`), and the `g(0)` slot (`availability()`, the three view-window functions, `g0()`). The fitting itself is done by `mrds::ddf()` and is described at its interface only. Effort, segmentation and the derivation of distances belong to `distsamp` and are described where they reach this package.

Line references are to the source as of this date (`R/sweep.R:103`). The architecture and the list of guards are in `docs/01-plan.md`; this chapter does not repeat the reasoning there, and explains the code as it now stands.

Terms used throughout:

Term	Meaning
flatfile	One row per detection, with `distance` or `distbegin`/`distend`. The shape `distsamp::detection_data()` returns.
exact	A point distance in `distance`. Fitted with the exact-distance likelihood.
binned	An interval in `distbegin`/`distend`. Fitted with the binned (multinomial) likelihood over `breaks`.
truncation	The right truncation width `w`. One value per sweep.
`left`	The left truncation distance: the start of the strip the fit describes. Zero when not set.
candidate	One row of a `model_set()`: a key, an optional adjustment series and order, and a scale formula.
sweep	Every candidate in a model set fitted to the same prepared data. A `dsfit_sweep`.
`p`	Average detection probability over the searched strip, `summary(ddf)$average.p`.
`esw`	Effective strip half-width, `p * (w - left)`.
`distance_source`	The `distsamp` column naming which record supplied a distance: `angle`, `exact`, `strip`, `circling` or `break_off`.

2. The pipeline at a glance

Step	Function	File	Produces
0	`detection_structure()` (optional)	`R/structure.R:58`	what the data can support, by question
0	`diagnose_sweep()` (optional)	`R/diagnose.R:42`	a printed report; nothing fitted
1	`check_mrds()`	`R/sweep.R:334`	an error if `mrds` is absent
2	`model_set()`	`R/model-set.R:85`	one row per candidate, with its `dsmodel` string

Step	Function	File	Produces
3	`prepare_distance_data()`	`R/prepare.R:66`	`data`, `binned`, `breaks`, `n_dropped`, `dropped`
3a	`check_circling()`	`R/prepare.R:201`	an error or warning for circling-derived distances
3b	`derive_breaks()`	`R/prepare.R:237`	cutpoints, for binned data without `breaks`
4	`mrds::ddf()` per candidate	`R/sweep.R:137`	`fits`, named by `model_id`; `NULL` for a failure
5	`build_selection_table()`	`R/sweep.R:212`	the table, sorted by AIC
5a	`fit_summary()`	`R/sweep.R:246`	`n_par`, `p`, `p_se`, `esw`, goodness of fit
5b	`fit_gof()`	`R/sweep.R:292`	`cvm_p`, `chisq_p`
6	`report_sweep()`, `print.dsfit_sweep()`	`R/sweep.R:306`, `R/print.R:2`	the message and the printed summary
7	`effect_estimates.ddf()`	`R/effect-estimates.R:105`	`g(x)` on a grid, with a delta-method band
8	`availability()`, `view_window*()`, `g0()`	`R/availability.R`, `R/g0.R`	a `dsfit_g0` for the abundance step

Steps 0 and 8 sit outside the fitting path. Step 0 runs the same guards as step 3 without fitting. Step 8 builds the correction the fit deliberately does not contain. The intended route is therefore:

```
flat <- distsamp::detection_data(segs, area = 5811)
diagnose_sweep(flat, model_set(c("hn", "hr", "gamma")), truncation = 2000)
sw   <- sweep_models(flat, model_set(c("hn", "hr", "gamma")), truncation = 2000)
selection_table(sw)
```

3. `prepare_distance_data()` — `R/prepare.R:66`

3.1 Argument checks and the distance type

```
has_point <- "distance" %in% names(data) && any(!is.na(data$distance))
has_bin   <- all(c("distbegin", "distend") %in% names(data)) &&
  any(!is.na(data$distbegin))
```

The type is decided by which columns carry values, not by which columns exist, because a `distsamp` flatfile always carries all three. Neither present is an error. Both present is an error: the exact and binned likelihoods differ, so AICs from the two cannot be ranked together, and a table holding both marks a survey-era boundary. The message names `distance_source` as the column to split on.

`left` must be non-negative and smaller than `truncation`.

3.2 Circling-derived distances — `check_circling()`, `R/prepare.R:201`

```
src    <- as.character(data$distance_source)
fitted <- !is.na(data$distance)
n_break <- sum(fitted & src %in% "break_off")   # error
n_circ  <- sum(fitted & src %in% "circling")    # warning
```

`distsamp::detection_data()` excludes detections made while circling unless `include_circling = TRUE`. When they arrive here they are one of two kinds, and the two are treated differently:

`distance_source`	What the distance is	Behaviour
`break_off`	Great-circle distance from where the aircraft left the line to where the record was logged. Not perpendicular to anything.	error
`circling`	The sighting's exact position projected onto the census line at the break-off. Perpendicular, but fixed minutes after detection.	warning, with the count

Only rows that would be fitted are counted. A circling row with no `distance` is dropped in Section 3.3 with the other missing distances. `%in%` is used rather than == so that `NA` sources compare as false. The check does nothing when `distance_source` is absent, so a flatfile from another source is unaffected.

3.3 Attrition by reason

```
gone_missing <- is.na(data$distance)
gone_beyond  <- !gone_missing & data$distance > truncation
gone_inside  <- !gone_missing & !gone_beyond & data$distance < left
```

Each dropped row is assigned to exactly one reason, in this order, so the three counts in `dropped` sum to `n_dropped`. The breakdown is returned because a total alone cannot distinguish a truncation that trimmed a tail from one that discarded half the survey. `diagnose_sweep()` prints it.

3.4 The binned branch

1. Rows with a missing `distbegin` or `distend` are removed.
2. Any `distend` of `Inf` is an error. Every `STRIP` code book has an open top bin, which no detection function can be fitted to.
3. Rows with `distend > truncation` or `distbegin < left` are removed. A bin is kept or dropped whole; it is never split.
4. `breaks` are derived (Section 3.6) if not given.
5. The top break must equal `truncation` (`R/prepare.R:145`).
6. The bottom break must equal `left`, or zero without it (`R/prepare.R:160`).

Steps 5 and 6 are the same guard at the two ends of the strip. A binned fit integrates the detection function over the bins, and `mrds` normalises it over `(left, width)`. A strip between the outermost break and either limit is effort with no bin to place detections in. The bottom guard is reached in two ways: derived breaks when the innermost bin had no detections, and a `left` that falls inside a bin, whose detections step 3 then drops. Both messages name the two remedies: move the limit to the break, or supply `breaks` that reach it.

3.5 The exact branch

```
keep <- !is.na(data$distance) & data$distance <= truncation
if (!is.null(left)) keep <- keep & data$distance >= left
```

Truncation is inclusive at both ends. An empty result is an error rather than an empty fit. `ensure_object()` (`R/prepare.R:257`) adds a sequential `object` column when absent, since `mrds` keys detections on it.

3.6 `derive_breaks()` — `R/prepare.R:237`

The unique `(distbegin, distend)` pairs are sorted by `distbegin`, and each pair's end must equal the next pair's start. A gap or an overlap means the intervals define no single set of cutpoints, and is an error naming the first interval before the gap. The cutpoints are the sorted unique edges.

4. `model_set()` — `R/model-set.R:85`

4.1 Validation and the grid

Keys are checked against `hn`, `hr`, `unif`, `gamma`; adjustment series against `cos`, `herm`, `poly`. `order` must be whole numbers. `formula` is a formula or a list of them. `expand.grid()` (`R/model-set.R:118`) crosses key, adjustment, order and formula index.

Two uniform-key candidates are removed with a warning. With no adjustment (`R/model-set.R:130`) it is a flat function, a strip transect, and fitting it as a candidate is almost never what was meant. With a covariate formula (`R/model-set.R:146`) the formula has nothing to act on: covariates enter through the scale parameter, which the uniform key lacks, and `mrds` ignores the formula rather than refusing it. Kept, the candidate would duplicate the uniform model without covariates under a covariate label.

4.2 The `dsmodel` string — `build_dsmodel()`, `R/model-set.R:206`

```
fn <- if (covariate) "mcds" else "cds"
paste0("~", fn, "(key = \"", key, "\", formula = ", formula,
       ", adj.series = \"", adjustment, "\", adj.order = ", order, ")")
```

A formula other than `~1` selects `mcds`. The string is stored in the table and passed to `stats::as.formula()` at fit time, so the model a row describes is readable without refitting. `label_model()` builds `model_id` as `key[+adjustment(orders)][ covariates]`, for example `hn+cos(2,3) bf`.

4.3 Cumulative adjustment orders — `adjustment_orders()`, `R/model-set.R:236`

```
if (adjustment == "cos") { start <- if (key == "unif") 1 else 2; step <- 1 }
else                     { start <- if (key == "unif") 2 else 4; step <- 2 }
if (is.na(max)) return(start)
if (max < start || (max - start) %% step != 0) return(NULL)
seq.int(start, max, by = step)
```

Each element of `order` is the highest order of one candidate, and the candidate carries every term of its series up to it, as `Distance::ds()` does (its internal `get_adj_orders()`). `order = 3` on `hn` with `"cos"` gives `adj.order = c(2, 3)`. With no `order`, each candidate has one term, the first of its series. The gamma key takes the `hn`/`hr` series; `Distance` does not offer it. An order outside the series is dropped with a warning naming it, rather than passed to `mrds`, where it failed.

Key	Cosine	Hermite, polynomial
`hn`, `hr`, `gamma`	2, 3, 4, …	4, 6, 8, …
`unif`	1, 2, 3, …	2, 4, 6, …

5. `sweep_models()` — `R/sweep.R:103`

5.1 Arguments and settings

`truncation` has no default and its absence is an error naming why: AIC compares models fitted to the same data, and truncation defines that data. The gamma key together with `left` warns, since both describe a blind spot beneath the platform.

```
meta <- list(width = truncation)
if (!is.null(left)) meta$left <- left
if (prepared$binned) { meta$binned <- TRUE; meta$breaks <- prepared$breaks }
```

The same `meta.data` is passed to every candidate, which is what makes the AICs comparable.

5.2 The fitting loop — `R/sweep.R:137`

```
fit <- try(mrds::ddf(dsmodel = stats::as.formula(models$dsmodel[i]),
                     data = prepared$data, method = "ds",
                     meta.data = meta), silent = TRUE)
if (inherits(fit, "try-error")) {
  failures <- c(failures, setNames(
    sub("\n.*", "", conditionMessage(attr(fit, "condition"))),
    models$model_id[i]))
  fits[i] <- list(NULL)
} else if (!identical(as.numeric(fit$ds$converge), 0)) {
  failures <- c(failures, setNames("did not converge", models$model_id[i]))
  fits[i] <- list(NULL)
} else {
  fits[[i]] <- fit
}
```

A candidate fails in one of two ways. `mrds` either throws, or warns and returns an object with `ds$converge != 0` whose summary has no `average.p`. Both are recorded in `settings$failures` under the model's ID, and both leave a `NULL` in `fits`. `fits[i] <- list(NULL)` is used rather than `fits[[i]] <- NULL`, which would delete the element, shorten the list, and move every later fit onto the wrong row of the table.

`mrds` draws random starting values. Whether a marginal candidate converges can therefore depend on the session's random state; a seed set before the sweep makes it repeat.

5.3 The selection table — `build_selection_table()`, `R/sweep.R:212`

One row per candidate, failed or not. `converged` is `!is.null(fit)`, which after Section 5.2 means converged. `vapply()` carries the names of `fits` onto every column, and they are removed with `unname()` so a column pulled out is a bare vector. `delta_aic` is `NA` throughout when every candidate failed (`R/sweep.R:234`), rather than `NA - Inf` with a warning from `min()`. Rows are ordered by AIC with failures last.

5.4 Per-fit quantities — `fit_summary()`, `R/sweep.R:246`

```
left <- if (is.null(s$left)) 0 else s$left
p    <- scalar(s$average.p)
esw  <- p * (scalar(s$width) - left)
```

`average.p` is the mean of `g(x)` over `(left, width)`, so the strip it scales is `width - left`. `scalar()` (`R/sweep.R:275`) returns `NA` for a field that `mrds` left empty or absent, so a single malformed summary cannot stop the table being built. `p_cv` is `p_se / p`, computed in the table.

5.5 Goodness of fit — `fit_gof()`, `R/sweep.R:292`

`mrds::ddf.gof(f, qq = FALSE)` is called once per fit. `cvm_p` is taken from `dsgof$CvM$p`, which `mrds` computes for exact distances only. `chisq_p` is taken from `chisquare$chi1$p`, which exists for both. For binned data the chi-square is over the survey's own bins; for exact data it is over cutpoints `mrds` chooses, and `cvm_p` is the test to read. A chi-square with no degrees of freedom left is `NA`, which is a converged model with no test available.

5.6 Reporting

`report_sweep()` sends one `rlang::inform()` message: fits out of candidates, detections, truncation, rows dropped, the IDs that did not fit, and the `g(0) = 1` assumption. `print.dsfit_sweep()` shows the top five converged rows with whichever goodness-of-fit column applies: chi-square when every `cvm_p` shown is `NA` and some `chisq_p` is not, CvM otherwise.

6. Pre-flight checks

6.1 `detection_structure()` — `R/structure.R:58`

Reports, per question, whether the table supports it: `TRUE`, `FALSE`, or `NA` where the structure is present but incomplete. Covariate candidates are the columns that vary and are not structural. The structural list is the `mrds` and flatfile columns (`object`, `distance`, `distbegin`, `distend`, `size`, `distance_source`, and the like) plus, when `narwcr` is installed, `narwcr::narwc_never_fill()`. Leg-level columns from `narwcr::narwc_fill_columns()` are labelled as survey conditions. Aliases from `narwcr::narwc_schema()` map two columns naming one variable to a canonical name, and a duplicate is reported; a `<NAME>_ORIGINAL` column kept by `narwcr::read_narwc(prefer_source = TRUE)` is excluded.

`perception_structure()` (`R/structure.R:229`) requires `observer` and `detected`, at least two observers, and an `object` that appears twice. Availability is always reported as unsupported: it cannot be estimated from distances.

6.2 `diagnose_sweep()` — `R/diagnose.R:42`

Runs in order: toolchain, `detection_structure()`, `prepare_distance_data()`, sample size, model set, blind spot, covariates. Each finding is printed as `ok`, `note`, `WARN` or `FAIL`; `WARN` and `FAIL` set the closing line to "Problems above". A guard error stops the run and returns what was reached. Guard warnings, such as the circling warning of Section 3.2, are caught with `withCallingHandlers()` and printed as `WARN` lines rather than escaping as R warnings after the report.

The blind-spot check fires when the nearest detection exceeds 2% of the truncation width. `onset_shape()` (`R/diagnose.R:285`) then compares the first of 20 equal bins above that distance with the modal bin; a ratio above 0.6 is classed as an abrupt edge, which `left` removes and no key function reproduces.

7. `effect_estimates.ddf()` — `R/effect-estimates.R:105`

7.1 Arguments

`interval` and `scale` are resolved by `resolve_choice()` (`R/effect-estimates.R:219`) rather than `match.arg()`, because `fancyfx::plotEffects()` forwards its own unresolved default vectors. `"auto"` becomes `"se"`; `"cri"` is refused. `var` must be `"distance"`. The grid runs from `left` (or 0) to `width`.

7.2 The averaged curve — `averaged_g()`, `R/effect-estimates.R:186`

```
design <- as.data.frame(dm)
if (!is.null(shape_dm)) design <- cbind(design, as.data.frame(shape_dm))
key    <- do.call(paste, c(unname(design), sep = "\r"))
rows   <- rep(match(unique(key), key), each = length(x))
obj$scale$dm <- dm[rows, , drop = FALSE]
obj$shape$dm <- shape_dm[rows, , drop = FALSE]
xs <- rep(x, times = n_u)
```

`mrds::detfct()` pairs distances with design-matrix rows element by element, recycling the shorter. Every (distance, covariate row) pair is therefore made explicit: the unique rows of the combined scale and shape design matrices are each repeated once per grid point, and the distances are repeated once per unique row. The result is reshaped to a grid-by-row matrix and averaged with weights equal to each row's frequency. An intercept-only model has one unique row. The shape matrix, which hazard-rate and gamma keys carry, is expanded with the scale matrix so the two stay the same length.

`covariate_object()` (`R/effect-estimates.R:233`) rebuilds the scale design matrix from the model's data with the `at` values substituted, so named covariates are fixed and the rest are averaged as observed.

7.3 The band — `delta_se()`, `R/effect-estimates.R:260`

The Jacobian of the averaged curve with respect to the parameters is taken by `numDeriv::jacobian()` and combined with `solve(model$hessian)`. The band is clamped to `[0, 1]`. Where the Hessian does not invert, the standard errors are `NA` and a warning is given, so the curve is still returned.

8. The g(0) slot

8.1 availability() — R/availability.R:136

Implements Laake et al. (1997):

$$a = \frac{E(s) + E(d)\left(1 - e^{-w/E(d)}\right)}{E(s) + E(d)}$$

per element of surface, with dive and window recycled. The standard error is the delta method on (surface, dive) with their covariance matrix, by numDeriv::grad() in availability_se() (R/availability.R:531). It is NA when no standard errors are given. Giving one of se_surface and se_dive without the other is an error. The result has the component shape key, component, value, se.

8.2 View windows

Function	Geometry	w(x)
view_window() (R/availability.R:271)	circular patch of radius r	2 sqrt(r^2 − x^2) / v
view_window_aerial() (R/availability.R:353)	side-window wedge, Ganley et al. (2019)	alpha + x tan(theta) / v, or alpha + slope ∗ x
view_window_vessel() (R/availability.R:477)	forward sector of half-angle theta from the bow	(sqrt(r^2 − x^2) − x / tan(theta)) / v, floored at 0

view_window_ship() is an alias for view_window_vessel(). In the vessel case the animal enters the view where it crosses the search radius and leaves where its bearing exceeds theta; at theta = 90 it leaves when abeam, and the window is half of view_window(). The 90-degree case is set to an exact zero offset because tan(pi/2) is finite in floating point.

8.3 g0() — R/g0.R:112

Component tables are stacked, with source added as NA when absent. The following are errors: a component with no name, a missing se, a value outside (0, 1], and a component whose keys differ from the others (R/g0.R:187). A missing availability or perception component is a warning. Per key (R/g0.R:224), the value is the product of the matching components and the CV is the root sum of squared component CVs, which is the delta method under independence. An unkeyed single-row component applies to every key.

9. Ordering constraints and invariants

Constraint	Where enforced	Consequence if broken
One truncation and one meta.data for every candidate	sweep_models(), R/sweep.R:126	AICs from different data are ranked together.
Guards before fitting	prepare_distance_data() called before the loop	mrds errors on mixed or open-ended data, or fits it silently.
Failed fits leave a NULL in place	fits[i] <- list(NULL)	Later models take the previous model's fit and row.

Constraint	Where enforced	Consequence if broken
Unconverged fits are failures	`R/sweep.R:156`	The table cannot be built; previously stopped the sweep.
`esw` uses `width − left`	`fit_summary()`	`esw`, and density, overstated by `left / (width − left)`.
Binned breaks span `(left, width)` exactly	`R/prepare.R:145`, `R/prepare.R:160`	Unaccounted effort at either end of the strip.
Shape and scale design matrices expanded together	`averaged_g()`	Hazard-rate and gamma curves fail or recycle.
Circling checked on fitted rows only	`check_circling()`	Rows dropped for no distance are reported twice.

10. Defects found and corrected in this audit

Defect	Correction
A candidate that `mrds` returned unconverged stopped the whole sweep: its summary has `average.p` of length 0, and `vapply()` in the table builder errored. Such a fit was also labelled `converged`.	Recorded as a failure, "did not converge", when `ds$converge != 0`. `scalar()` makes empty summary fields `NA`. `delta_aic` no longer calls `min()` on an all-`NA` vector. Tests in `tests/testthat/test-sweep.R`.
`effect_estimates_ddf()` failed for intercept-only hazard-rate and gamma fits ("non-conformable arguments") and recycled for their covariate fits: only the scale design matrix was expanded to the grid.	Shape design matrix expanded alongside it. Tests check that the curve integrates to `average.p` for both keys.
`esw` was `p * width` under left truncation. At `left = 30`, `truncation = 400`, the half-normal `esw` was 118.4; the correct value is 109.5.	`p * (width − left)`. The left-truncation snapshot updated; AIC, `p` and ranking unchanged.
Binned data whose lowest break exceeded `left` (or zero) was fitted over `(left, width)` with no bin covering the inner strip. A `left` inside a bin dropped that bin's detections silently.	Refused, with both remedies named. Tests in `tests/testthat/test-prepare.R`.
The bins-do-not-tile error named interval `nrow(bins) − sum(gaps)`: "interval 4" for a gap after interval 2.	Names `which(gaps)[1]`.
Circling-derived distances were fitted without comment, and `distance_source` was offered as a detection covariate.	`check_circling()` (Section 3.2); `distance_source` added to the structural columns.
A uniform key with a covariate formula was accepted, and `mrds` ignored the formula: `unif+cos1 bf` and `unif+cos1` had the same parameter count (1) and AIC (1633.22).	Dropped with a warning by `model_set()`, as the uniform key with no adjustment already was. Test in `tests/testthat/test-model-set.R`.
Adjustment orders were one term per candidate: `order = 3` fitted the order-3 cosine term alone, unlike `Distance::ds()`. Orders outside a key's series (`hn+herm2`, `unif+herm3`) were passed to `mrds` and failed.	Cumulative, by `adjustment_orders()` (Section 4.3); out-of-series orders dropped with a warning. On the adjustment snapshot `unif+cos` with orders 1 and 2 has AIC 2178.35 against 2379.98 for the order-2 term alone.
The comment on `delta_se()` said it returned zeros when the Hessian does not invert. It returns `NA`.	Comment corrected.

11. Limitations

- `onset_shape()` uses `distbegin` for binned data, where a 20-bin histogram of bin starts carries little information about the onset. Its verdict is meaningful for exact distances only.
- Convergence of marginal candidates depends on the random state (Section 5.2). The sweep does not set a seed.
- `prepare_distance_data()` refuses a flatfile carrying both `distance` and `distbegin`/`distend` values, including one in which `distance` is the interval midpoint.
- `g0()` rejects a vector of availabilities with no `key` with a message about mixed keyed and unkeyed rows, which does not describe the cause.
- The vignettes and tests use synthetic data. No real NARWC extract was swept in this audit.

12. Verification performed

Check	Result
`R CMD check --no-manual`, before fixes	0 errors, 0 warnings, 1 note (top-level pkgdown files); all tests pass
Unconverged candidate, before fix	`hr` on three distances: `ds$converge = 1`; `fit_summary()` returned `p = numeric(0)`; the sweep stopped in `vapply()`
Hazard-rate and gamma curves, before fix	Both stopped with "non-conformable arguments"; half-normal correct (mean 0.3509 against `average.p` 0.3494 on a 100-point grid)
`esw` under `left = 50`, `truncation = 400`, before fix	104.9 reported; `p * (width - left)` = 91.8
Bins 100–400 with no `left`, before fix	Fitted with `left = 0`, `int.range = (0, 400)`
`R CMD check --no-manual`, after fixes	0 errors, 0 warnings, 1 note (unchanged); 446 expectations pass, 0 fail, 0 test warnings
`hn+cos(2,3)` against `Distance::ds(order = c(2, 3))`	AIC 2179.705 both; 3 parameters
Uniform key with a covariate, before fix	`unif+cos1 bf` and `unif+cos1`: 1 parameter each, AIC 1633.22 each

13. Planned work

1. Add the pkgdown files to `.Rbuildignore`, which clears the remaining note.
2. Make `g0()`'s message name a missing `key` when every row of a component is unkeyed.
3. Sweep a real extract end to end, from `distsamp::detection_data()`.

14. References

Buckland, S.T., Anderson, D.R., Burnham, K.P., Laake, J.L., Borchers, D.L. and Thomas, L. (2001) *Introduction to Distance Sampling: Estimating Abundance of Biological Populations.* Oxford University Press. Truncation, sample size, goodness of fit, and effective strip half-width.

Miller, D.L., Rexstad, E., Thomas, L., Marshall, L. and Laake, J.L. (2019) Distance sampling in R. *Journal of Statistical Software* 89(1):1–28. doi:10.18637/jss.v089.i01. The `mrds` interface.

Becker, E.A. and Quang, P.X. (2009) A gamma-shaped detection function for line-transect surveys with mark-recapture and covariate data. *Journal of Agricultural, Biological and Environmental Statistics* 14:207–223. doi:10.1198/jabes.2009.0013. The gamma key.

Laake, J.L., Calambokidis, J., Osmek, S.D. and Rugh, D.J. (1997) Probability of detecting harbor porpoise from aerial surveys: estimating g(0). *The Journal of Wildlife Management* 61:63–75. doi:10.2307/3802415. The availability formula of Section 8.1.

Laake, J.L. and Borchers, D.L. (2004) Methods for incomplete detection at distance zero. In S.T. Buckland et al. (eds) *Advanced Distance Sampling*, pp. 108–189. Oxford University Press. Why the `g(0)` components are separate.

Ganley, L.C., Brault, S. and Mayo, C.A. (2019) What we see is not what there is: estimating North Atlantic right whale *Eubalaena glacialis* local abundance. *Endangered Species Research* 38:101–113. doi:10.3354/esr00938. The aerial view window of Section 8.2.